

פרויקט גמר – GatherUp שלב ד': שכבת ההגשה ושירותי רשת (Web API)

בשלב זה נחליף את פרויקט הקונסול הלימודי בשכבת הגשה מודרנית מבוססת פרוטוקול HTTP. המטרה היא לחשוף את כל היכולות הלוגיות של המערכת שבניתן בשלבים הקודמים כנקודות קצה ברשת, תוך שמירה על ארכיטקטורה נקייה, ניהול שגיאות ריכוזי ואבטחת מידע.

חלק 1: הגדרת הפרויקט ב-Solution

1. הוסיפו ל Solution-הקיים פרויקט חדש מסוג **ASP.NET Core Web API** בשם **GatherUp.API**.
2. בהגדרות יצירת הפרויקט ודאו כי סימנת את האפשרויות הבאות:
 - **Framework:** גרסת ה .NET-העדכנית שלכן (בהתאמה לשאר הפרויקטים).
 - **Use controllers:** חובה לסמן ב V-כדי לעבוד במבנה קונטרולרים תקני ולא כ-Minimal API.
 - **Enable OpenAPI support / Swagger:** השאירו מסומן לצורך דף בדיקות ויזואלי.
3. **ניהול קשרי פרויקטים:** פרויקט ה API-הוא שכבת העל הניהולית של המערכת. הגדירו בו רפרנסים (References) לכל שלושת הפרויקטים הקודמים ב Solution-שלכן.
4. מחקו את קוד הדוגמה הפיקטיבי שהגיע עם תבנית המערכת.

חלק 2: חלוקת הקונטרולרים (החלונות והמסכים)

- אין לרכז את כל הפעולות בקונטרולר אחד. עליכן ליצור מערך קונטרולרים הגיוני המבוסס על המחלקות הלוגיות שפיתחתן בשכבת ה BL:-
- **AuthController:** ינהל את מסך הכניסה של האפליקציה, יבצע אימות נתונים והנפקת טוקנים מאובטחים.
 - **EventsController:** ינהל את הגדרות האירוע, עריכת פרטים דינמיים, תיאום לוחות זמנים והרשמה לערוצי עדכונים.
 - **ParticipantsController:** ינהל את הצגת המשתתפים, עדכוני אישורי הגעה, הפצת הזמנות המונית ושליחת תזכורות.
 - **PersonController:** ינהל את הוספת משתמש חדש ועריכה שלו.
 - **FinancialController:** ינהל את מודול הספקים, הוספת חובות, העלאת קבלות דיגיטליות ודוח סיכום תקציב דינמי.
 - **PollsController:** ינהל את מנוע הסקרים, יצירת שאלונים מקדימים, והפקת מסקנות באחוזים.
 - **צרו קונטרולרים נוספים על פי הצורך-** כל עוד יש היגיון לוגי.

חוקי מבנה קבועים: כל הקונטרולרים יירשו ממחלקת הבסיס **ControllerBase**, יעוטרו ב **[ApiController]**-ויקבלו את מופעי ה **Managers**-הרלוונטיים משכבת ה BL-אך ורק באמצעות **הזרקת תלויות בבנאי המחלקה**.

חלק 3: ניהול חריגים (Global Exception Handling)

כדי למנוע קוד מסורבל ושכפולים, אין לכתוב בלוקים של try-catch ידניים בתוך הקונטרולרים. הניהול יבוצע בצורה ריכוזית:

1. **חריגים מותאמים אישית:** הגדירו תחת פרויקט הליבה שלכן מחלקות חריגים עסקיים חדשות לבחירתכן, המייצגות כשלים לוגיים במערכת כגון:

- חריג עבור אירוע או משתתף או כל רשומה לא קיימת.

○ חריג עבור ניסיון עריכה או מחיקה של קבלה פיננסית נעולה .

○ חריג עבור משתנים לא תקינים ועוד

2. **זריקה ואכיפה:** ודאו כי שכבת ה-BL-מזהה את כשלי הלקוח ויודעת לשגר את השגיאות באמצעות מילת המפתח. throw.

3. **Custom Middleware מרכזי:** הקימו בפרויקט ה-API-מחלקה המשמשת כ-Middlewares מרכזי לטיפול בשגיאות .

המידלור יממש את מוסכמת המבנה האסינכרונית הנדרשת, יעטוף את כל הצינור בבלוק try-catch גלובלי, ויפרש את סוגי החריגים שלכן לקודי סטטוס HTTP תקיניים (סטטוס 400 עבור שגיאות קליינט עסקיות, וסטטוס 500 עבור שגיאות שרת פנימיות או בעיות בדיסק ה-XML) .

חלק 4: מנגנון האוטנטיקציה וההרשאות - (JWT)

חלק יצירת טוקן וניהולו הוא אתגר- מספיק לשלוח מצד לקוח בכל גישה את ID לקוח וכך לבדוק בעזרת פונקציה יעודית האם יש לו הרשאות לבצע את מה שהוא מנסה. (כגון פעולות מנהל)

עליכן להגן על השרת מפני גישות לא מורשות באמצעות שילוב של טוקנים חתומים ו-Attributes:

1. **הנפקת טוקן בלוגאין:** בתוך ה-AuthController מממשו נקודת קצה לקבלת שם משתמש וסיסמה. אם האימות מול ה-Repository מצליח, השרת ינפיק JWT Token חתום המכיל Claims קבועים כגון מזהה המשתמש, שמו ותפקידו הגלובלי במערכת .

2. **הפרדת תפקידים מול אבטחת הקשר:**

○ עבור פעולות התלויות בתפקיד קבוע, השתמשו ב-Attribute המובנה [Authorize(Roles = "...") מעל נקודות הקצה הרלוונטיות.

○ עבור פעולות התלויות באירוע ספציפי (למשל: רק מי שהוא מארגן האירוע הנוכחי רשאי לערוך אותו), לא ניתן להסתמך על תפקיד גלובלי בטוקן. עליכן לחלץ את ה-ID של המשתמש מהטוקן בתוך קוד הפונקציה, לפנות לשכבת ה-BL לבדוק זכויות בעלות מול מאגר הנתונים, ולהחזיר סטטוס Forbid (שגיאת 403) אם המשתמש אינו מורשה נקודתית .

3. **החרגות:** השתמשו באטריביוט [AllowAnonymous] כדי להשאיר נקודות קצה חיוניות פתוחות לציבור הרחב ללא צורך בטוקן קודם, כגון הצגת פרטי אירוע כלליים או דף הלוגאין עצמו .

חלק 5: שילוב דפי HTML סטטיים (תיקיית wwwroot)

אתגר- אין חובה ליצור צד לקוח

המערכת צריכה לשמש גם כשרת קבצים מהיר עבור דפי ממשק המשתמש שלכן :

1. הקימו תיקייה ייעודית בשם **wwwroot** בשורש פרויקט API- (באותו הגובה של Program.cs) .

2. מקמו בתוכה קובצי HTML, תמונות וסטייל קבועים המשמשים כמסכי המערכת .

3. חברו בתוך קובץ השרת הראשי את המידלור המובנה app.UseStaticFiles() במיקום הנכון בצינור, כדי שהשרת יגיש את הדפים האלו ישירות לדפדפן ויבצע עצירת צינור (Short-Circuiting) מבלי לבזבז משאבי מערכת בניתובים לקונטרולים.

חלק 6: קביעת הצינור והזרקת תלויות ב-Program.cs

פתחו את קובץ השרת הראשי ובצעו את הגדרות התשתית הבאות :

1. **רישום שירותים: (Services)** רשמו ב-Container את מימושי ה-Repository מבוססי ה-XML שלכן, את מחלקות ה-Managers של שכבת ה-BL, ואת שירותי האבטחה הנדרשים .

2. **סדר פס הייצור: (The Pipeline)** סדרו את רכיבי המידלור בצינור בצורה קשיחה ומדויקת על פי הלוגיקה הבאה:

- מידלור ניהול השגיאות הגלובלי יוצב ראשון כדי לעטוף את כל התהליך.
- מידלור הקבצים הסטטיים יוצב לפני מערכת הניתוב.
- ה Middlewares-המובנים של האבטחה יוצבו בסדר המחייב: קודם UseAuthentication (אימות) ורק לאחר מכן UseAuthorization (הרשאה), רגע לפני החיבור הסופי לקונטרולים.

חלק 7: סימולציה ובדיקת הדיסק

1. הפעילו את השרת והשתמשו בדף ה Swagger או בכלי בדיקות חיצוני כדי לבצע תהליך לוגאין מלא ולקבל את ה Token.
2. העתיקו את ה Token והשתמשו בו כדי להריץ את פעולות ההוספה, העדכון והמחיקה החסומות שלכן.
3. **בדיקת הדיסק הפיזי:** גשו לתיקיית הריצה של השרת במחשב שלכן, פתחו את קובצי ה XML וודאו כי הפעולות שהרצתן דרך נקודות הקצה ברשת (כגון הוספת משתתפים, עדכוני חובות לספקים או שינוי בחירות בסקרים) חלחלו ונשמרו בהצלחה בדיסק בצורה מסונכרנת, נקייה ומנותקת לחלוטין.
4. **אתגר -** צרו דפי HTML עבור צד הלקוח שלכן והשתמשו בהם לכל פעולות הפרויקט.

חלק 8: ההגשה:

1. עליכן להכין קבצי XML עם לפחות 2 אירועים עם סקרים וכמה משתתפים, כך שלמשתתף מסוים אותו נבחן יהיה אירוע בו הוא המנהל ואירוע בו הוא משתתף.
2. מי שבחרת שלא ליצור צד לקוח חייבת להכין רשימה מסודרת של פעולות ב SWAGGER שיידמו את מסכי צד לקוח, לדוגמא- מה הניתובים כדי להכנס למערכת -לוגאין ואז להוסיף אירוע וליצור סקרים ולקבל את סיכום המצב בסקר, וכך הלאה לפי האיפיון של הפרויקט והמסכים שבשלב 3.
3. חובה להעלות את כל הפרויקט לגיטהב כך שנוכל להוריד ולקמפל מחדש, חובה לבדוק לפני ההגשה שהקוד כולו מתקמפל, לא לשכוח להעלות את קבצי ה XML וקבלי ה HTML למי שבחרה ליצור צד לקוח.
4. נא להגיע להגשה כשאתן מכירות את הקוד שלכן, אני בהחלט אוכל לשאול איפה הקוד שמבצע משהו מסויים, זה בסדר לא לדעת איפה, אבל חובה לדעת איך למצוא.
5. כמו כן אוכל לבקש להסביר לי קטעי קוד ושוב, אין בעיה לא לזכור אבל חובה לדעת להסביר.
6. בנוסף בבקשה להכין תהליך הצגה מסודר של הפרויקט, בו אתן מציגות לי את כל אופציות המערכת מכניסה ועד הניהול השלם של הפרויקט גם כמנהל וגם כמשתתף.
7. נא לחלק את ההגשה בצורה שווה בין שתי הבנות המגישות לפי צורת העבודה שלכן
8. יש להכניס את שמות בנות הזוג באקסל ההגשות בשעה המתאימה לכן-
9. שימו לב- חובה על כל זוג להתעדכן עם הזוג אחד לפני הקודמים להם – כשהן מסיימות להגיש את הפרויקט- ואז להופיע 20 דקות לאחר מכן
9. איחור של יותר מ 5 דקות- מוריד 5 דקות מההגשה

ריכוז האתגרים:

1. יצירת שרת למשלוח מיילים- 10 אחוז
2. ניהול טוקנים – 10 אחוז
3. יצירת צד לקוח – 15 אחוז